SHADOWFOX | MACHINE LEARNING ENGINEER INTERNSHIP

Machine Learning Engineer Internship

Final Report

Three projects across three levels: Boston House Price Prediction, Car Selling Price Prediction and ReviewMind AI

BEGINNER	INTERMEDIATE	ADVANCED
Boston House Price Prediction	Car Selling Price Prediction	ReviewMind AI

Student name	Samarth Jain
Institute	Sage University, Indore
Enrolment number	25ADV3ARI0039 / 34775
Programme	B.Tech AI-ML (IAC)
Internship role	Machine Learning Engineer Intern
Organisation	ShadowFox (Virtual Internship Program)
ShadowFox Program ID	SF-PID-2026-W7613
Internship duration	1 November 2026 – 30 November 2026 (1 month)

Acknowledgement

I would like to thank **ShadowFox** for the opportunity to take part in the Virtual Internship Program as a Machine Learning Engineer Intern. The program's structure of one project level per week, from Beginner through Intermediate to Advanced, gave the internship a clear progression and allowed each project to build on the skills of the one before.

I am grateful to the ShadowFox program team for organising the internship, and I also thank the faculty and staff of **Sage University, Indore**, for their support.

I am thankful to the authors and maintainers of the open-source tools and public datasets used in these projects, including Python, scikit-learn, Pandas, Matplotlib, Seaborn, PyTorch, Hugging Face Transformers and Streamlit, and to my family and friends for their encouragement.

Samarth Jain
B.Tech AI-ML (IAC), Sage University, Indore
Machine Learning Engineer Intern, ShadowFox

Abstract / Executive Summary

This report documents the work completed during the one-month ShadowFox Machine Learning Engineer Internship (1–30 November 2026). The internship was organised as three project levels. At each level a complete machine-learning workflow was carried out, from data inspection and preparation through modelling and evaluation to documentation, and the projects grew in scope from a single-notebook regression study to an interactive NLP application.

Beginner — Boston House Price Prediction. Five regression models were trained on 13 housing and neighbourhood features (506 records, 120 missing values imputed) to predict the median home value MEDV. On a 102-record test set, Random Forest was selected as the best overall model with MAE 2.2140, MSE 9.1504 and R^2 0.8752; Support Vector Regression had the lowest MAE (2.0810) and Linear Regression was the weakest baseline (R^2 0.6591).

Intermediate — Car Selling Price Prediction. After removing two duplicates (299 records) and engineering a Car_Age feature, five baseline models and two tuned ensembles were compared on a 60-record test set. The baseline Gradient Boosting Regressor performed best (MAE 1.1349, RMSE 2.5232, R^2 0.7530), with Linear Regression practically tied on R^2 (0.7528). Present_Price accounted for about 89.85% of the model's feature importance, followed by Car_Age (about 7.49%). The final preprocessor and model were saved with Joblib.

Advanced — ReviewMind AI. A feedback-intelligence platform combining sentiment analysis, emotion detection, topic modelling and an interactive Streamlit application with a local rule-based assistant, Ask ReviewMind. On the held-out IMDb test split of 9,917 reviews, TF-IDF with Logistic Regression reached 90.52% accuracy, 90.69% F1 and 0.9663 ROC-AUC, while a DistilBERT model fine-tuned on only 4,000 reviews for two epochs on a CPU reached 87.56% accuracy, 87.30% F1 and 0.9501 ROC-AUC. The backend integration notebook recorded 10 of 10 checks passed.

Table 1. *Summary of the three internship projects*

Level	Project	Task	Best model / headline result
Beginner	Boston House Price Prediction	Regression, 506 records	Random Forest: R^2 0.8752, MSE 9.1504, MAE 2.2140
Intermediate	Car Selling Price Prediction	Regression, 299 records	Gradient Boosting (baseline): R^2 0.7530, RMSE 2.5232, MAE 1.1349
Advanced	ReviewMind AI	Sentiment, emotion, topics; Streamlit app	TF-IDF + Logistic Regression: accuracy 90.52%, F1 90.69%, ROC-AUC 0.9663

Across the three projects, the consistent lessons were to compare models on identical held-out data, to state the training conditions behind every result, and to report limitations openly. Each report therefore records caveats alongside its results: small test sets, preprocessing before the split in the Boston project, the exploratory use of the test set and mixed vehicle types in the Car project, and single-domain training data in ReviewMind AI.

Keywords: machine learning, regression, house price prediction, used-car price prediction, sentiment analysis, emotion detection, topic modelling, DistilBERT, Streamlit.

Table of Contents

Acknowledgement 2

Abstract / Executive Summary 3

1. Introduction 5
- 1.1 Overview of the internship 5
- 1.2 Objectives 5
- 1.3 Skills and technologies developed 5

2. Beginner-Level Project: Boston House Price Prediction 7
- 2.1 Problem statement 7
- 2.2 Dataset description 7
- 2.3 Data preprocessing and exploratory data analysis 7
- 2.4 Models implemented 10
- 2.5 Evaluation metrics and actual results 10
- 2.6 Visualisations of model performance 11
- 2.7 Conclusion 12

3. Intermediate-Level Project: Car Selling Price Prediction 13
- 3.1 Problem statement 13
- 3.2 Dataset description 13
- 3.3 Data preprocessing and feature engineering 14
- 3.4 Models implemented 16
- 3.5 Model comparison and actual results 17
- 3.6 Feature importance 18
- 3.7 Conclusion 19

4. Advanced-Level Project: ReviewMind AI 20
- 4.1 Problem statement 20
- 4.2 Project objectives 20
- 4.3 System architecture and workflow 20
- 4.4 Dataset and preprocessing 22
- 4.5 Sentiment analysis 23
- 4.6 Emotion detection 23
- 4.7 Topic modelling 24
- 4.8 Ask ReviewMind assistant 24
- 4.9 Interactive dashboard and visualisations 25
- 4.10 Evaluation results 25
- 4.11 Deployment and limitations 28
- 4.12 Conclusion 29

5. Overall Learning Outcomes 30
- 5.1 Machine learning concepts 30
- 5.2 Data analysis 30
- 5.3 Model evaluation 30
- 5.4 Python development 30
- 5.5 Application development 30
- 5.6 Deployment and GitHub 31

6. Challenges Faced and Solutions 32

7. Future Improvements 33
- 7.1 Project-specific improvements 33
- 7.2 Cross-cutting improvements 33

8. Overall Internship Conclusion 34

9. References 35

1. Introduction

1.1 Overview of the internship

The ShadowFox Virtual Internship Program is a one-month program (1–30 November 2026) in which the intern works on real-world style projects at three levels. As a Machine Learning Engineer Intern, the author completed one project per level. According to the program timeline, Week 1 was devoted to the Beginner project, Week 2 to the Intermediate project, and Weeks 3 and 4 to the Advanced project together with project submission and a question-and-answer session.

Table 2. *Internship structure and the project completed at each level*

Week	Program level	Project completed	Main deliverables
Week 1	Beginner	Boston House Price Prediction	Notebook, project report, presentation
Week 2	Intermediate	Car Selling Price Prediction	Notebook, saved model (.pkl), project report, presentation
Weeks 3–4	Advanced, plus submission Q&A	ReviewMind AI	Notebooks, Streamlit application, GitHub repository, project report, presentation

1.2 Objectives

- Apply the supervised machine-learning workflow end to end: data inspection, cleaning, exploratory analysis, modelling, evaluation and reporting.
- Train several models on the same task and compare them fairly using appropriate metrics (MAE, MSE, RMSE and R^2 for regression; accuracy, precision, recall, F1-score and ROC-AUC for classification).
- Progress from classical regression on small tabular data to natural-language processing with both classical and transformer models.
- Package a trained model for reuse (Car project) and build an interactive, modular application around several models (ReviewMind AI).
- Document every project honestly, including limitations, in reports and a presentation, and use Git and GitHub for version control where applicable.

1.3 Skills and technologies developed

Table 3. *Technologies used across the three projects*

Area	Tools and techniques	Used in
Programming and notebooks	Python and Jupyter Notebook (all projects); Visual Studio Code (ReviewMind AI)	All projects
Data handling	Pandas, NumPy; imputation, duplicate removal, outlier screening, encoding, feature engineering	All projects
Visualisation	Matplotlib, Seaborn; histograms, box plots, heatmaps, scatter plots, confusion matrices, ROC curves	All projects

Area	Tools and techniques	Used in
Classical machine learning	Scikit-learn: Linear Regression, Decision Tree, Random Forest, Gradient Boosting, SVR, TfidfVectorizer, Logistic Regression, NMF, Pipeline, ColumnTransformer, RandomizedSearchCV	All projects
Deep learning and NLP	PyTorch (CPU build), Hugging Face Transformers; fine-tuned DistilBERT; pretrained DistilRoBERTa for emotion detection	ReviewMind AI
Application development	Streamlit; modular Python backend (src/ modules); CSV and PDF export; integration testing in a notebook	ReviewMind AI
Model persistence	Joblib (.pkl) and Hugging Face save_pretrained artifacts	Car; ReviewMind AI
Version control	Git and GitHub	Boston House Price Prediction; ReviewMind AI

2. Beginner-Level Project: Boston House Price Prediction

BEGINNER LEVEL
Week 1 · Project 1 of 3

Boston House Price Prediction

2.1 Problem statement

Given 13 attributes of a Boston-area tract (crime rate, zoning, business acreage, river proximity, air pollution, number of rooms, building age, distance to employment centres, highway access, tax rate, pupil-teacher ratio, a demographic-derived index and the percentage of lower-status population), predict the median value of owner-occupied homes, **MEDV**. Because MEDV is continuous, this is a supervised regression problem. The scope was one tabular dataset and a notebook-based Python workflow with five regression models and a single held-out test set; no systematic tuning, cross-validation, saved model or deployed application was included.

2.2 Dataset description

The file HousingData.csv contains **506 rows and 14 columns**: 13 input features and the target MEDV, with no duplicate rows. Six columns (CRIM, ZN, INDUS, CHAS, AGE and LSTAT) each have 20 missing values (3.95% of rows per column, 120 in total); the other eight columns are complete. The unit of MEDV is not stated in the notebook; in the original dataset documentation it is thousands of US dollars, and the report treats it as a relative price value.

Table 4. *Boston dataset columns and target statistics*

Item	Details
Features (13)	CRIM, ZN, INDUS, CHAS, NOX, RM, AGE, DIS, RAD, TAX, PTRATIO, B, LSTAT
Target	MEDV: mean 22.53, median 21.20, standard deviation 9.20, minimum 5.00, maximum 50.00
Missing values	CRIM, ZN, INDUS, CHAS, AGE, LSTAT: 20 each (120 in total)
Capped target	16 records sit exactly at the maximum of 50.00, suggesting the target is capped
Class balance of CHAS	93.00% of non-missing values are 0 and 7.00% are 1

2.3 Data preprocessing and exploratory data analysis

Preprocessing

- **Inspection and duplicates:** shape (506, 14), data types and descriptive statistics were checked; no duplicates were found.
- **Missing-value imputation:** CRIM, ZN, INDUS, AGE and LSTAT were filled with each column's median; CHAS was filled with its mode (0). Zero missing values remained afterwards.
- **Outlier screening:** the IQR rule on MEDV (Q1 = 17.025, Q3 = 25.0) gives bounds of 5.06 and 36.96 and flags 40 potential outliers. They were inspected but kept because unusual houses can be valid observations.
- **Split:** 80/20 with random_state = 42, giving 404 training and 102 test records. SVR was trained in a pipeline with StandardScaler; the other models used unscaled features.

Order of operations, as documented in the project report

Imputation was applied to the full dataset before the train/test split, which can leak a small amount of test-set information into training. A supplementary check that fitted the imputers on the 404 training records only changed the test metrics only in the third or fourth decimal place and left the model ranking unchanged (see Section 2.6). Fitting preprocessing on training data only remains the recommended practice.

Exploratory data analysis

MEDV is concentrated between about 17 and 25, with a median of 21.2 and a right-hand tail; the spike at 50.0 is consistent with a capped target.

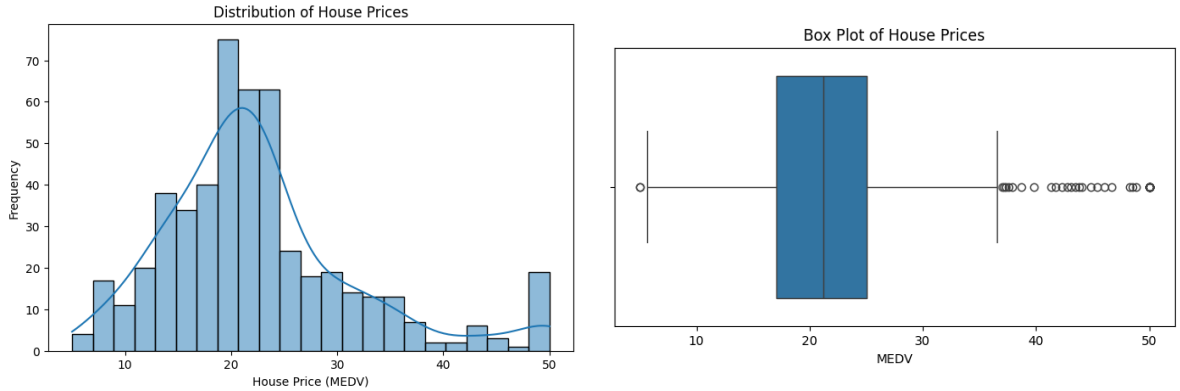


Figure 1. *Distribution and box plot of house prices (MEDV).*

Correlations with MEDV (after imputation) show that the average number of rooms **RM** is the strongest positive relationship (r = +0.695), while **LSTAT** (r = −0.723) and **PTRATIO** (r = −0.508) are the strongest negative ones. Correlation measures linear association and does not establish causation.

Table 5. *Strongest correlations of features with MEDV*

Negative feature	r	Positive feature	r
LSTAT	−0.723	RM	+0.695
PTRATIO	−0.508	ZN	+0.362
INDUS	−0.476	B	+0.333
TAX	−0.469	DIS	+0.250
NOX	−0.427	CHAS	+0.184
CRIM	−0.384		
RAD	−0.382		
AGE	−0.378		

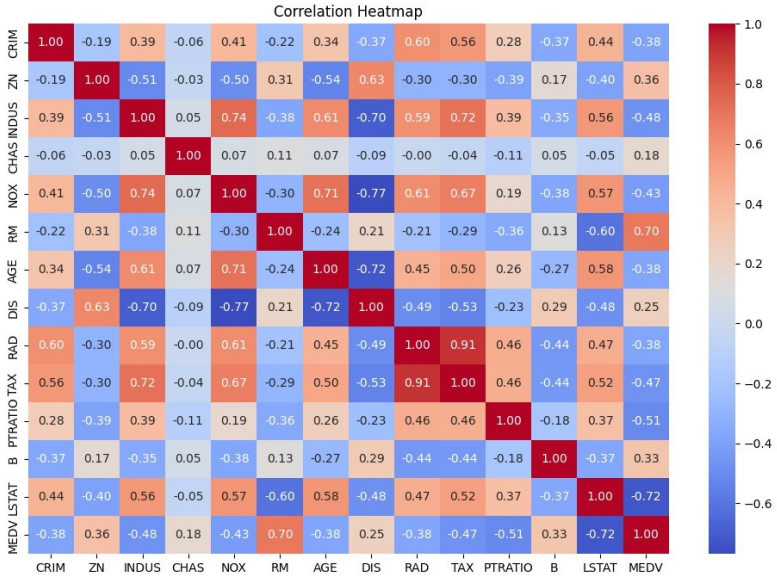


Figure 2. *Correlation heatmap of all variables including MEDV.*

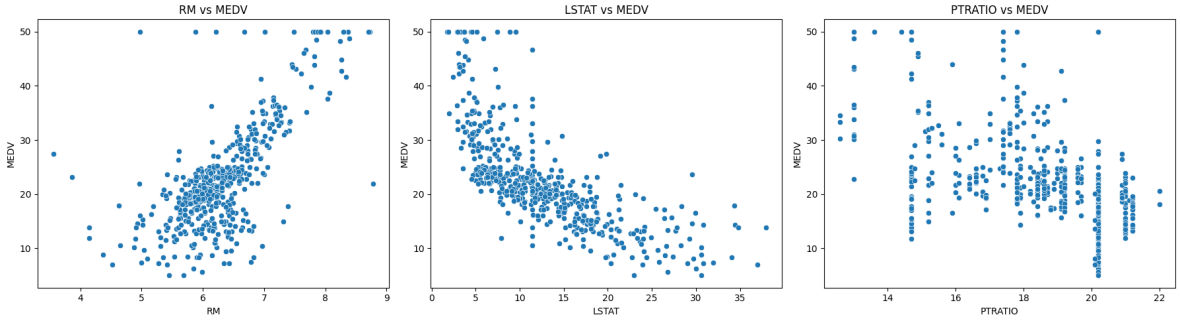


Figure 3. *Scatter plots of MEDV against RM, LSTAT and PTRATIO.*

MEDV rises with RM, falls as LSTAT increases in a curved, non-linear way, and tends to be lower at high PTRATIO. The curved LSTAT relationship helps explain why non-linear models later outperform Linear Regression. Several predictors are also strongly correlated with each other, notably RAD and TAX (0.91), NOX and DIS (−0.77), INDUS and NOX (0.74), AGE and DIS (−0.72) and INDUS and TAX (0.72), which makes linear-model coefficients harder to interpret.

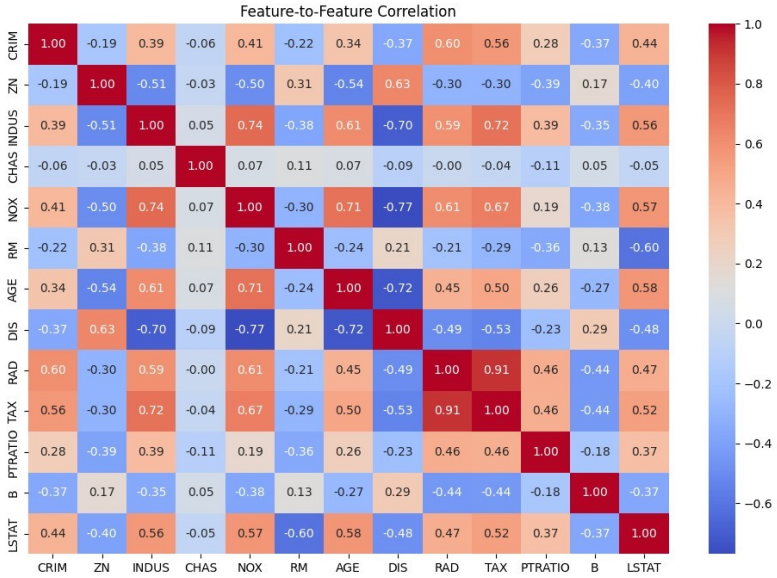


Figure 4. *Feature-to-feature correlation heatmap (MEDV excluded).*

2.4 Models implemented

Five regression models were trained on the 404 training records with manually chosen hyperparameters; no systematic tuning or cross-validation was performed.

Table 6. *Boston model configurations*

Model	Configuration
Linear Regression	Default scikit-learn settings
Decision Tree Regressor	max_depth = 4, random_state = 42
Gradient Boosting Regressor	n_estimators = 100, learning_rate = 0.05, max_depth = 2, random_state = 42
Random Forest Regressor	n_estimators = 100, max_depth = 5, random_state = 42
Support Vector Regression	Pipeline of StandardScaler and SVR(kernel = “rbf”, C = 10, epsilon = 0.1)

The Linear Regression baseline has an intercept of 28.17; its largest coefficients are NOX (−15.98), RM (+4.72), CHAS (+3.28), DIS (−1.51) and PTRATIO (−0.89), although the features are on different scales and correlated with each other.

2.5 Evaluation metrics and actual results

MAE is the average absolute prediction error; **MSE** is the average squared error and penalises large mistakes more strongly; **RMSE** is the square root of MSE in the units of the target; and **R^2** is the proportion of variance in MEDV explained by the model (higher is better). Results on the 102-record test set are shown below; RMSE is derived as √MSE.

Table 7. *Boston test-set results (sorted by R^2)*

Model	MAE	MSE	RMSE	R^2
Random Forest	2.2140	9.1504	3.0249	0.8752
Decision Tree	2.7776	12.4201	3.5242	0.8306
Gradient Boosting	2.3624	12.4420	3.5273	0.8303
SVR	2.0810	13.9944	3.7409	0.8092
Linear Regression	3.1487	24.9994	4.9999	0.6591

Random Forest achieved the lowest MSE (9.1504) and the highest R^2 (0.8752), explaining about 87.5% of the variance in MEDV on the test set. SVR achieved the lowest MAE (2.0810) but a higher MSE (13.9944), meaning it makes some larger errors. Decision Tree and Gradient Boosting are practically tied, and Linear Regression is clearly the weakest. Random Forest was selected as the best overall model because it leads on two of the three metrics and has a competitive MAE. With only 102 test records, differences between close models should not be over-interpreted.

2.6 Visualisations of model performance

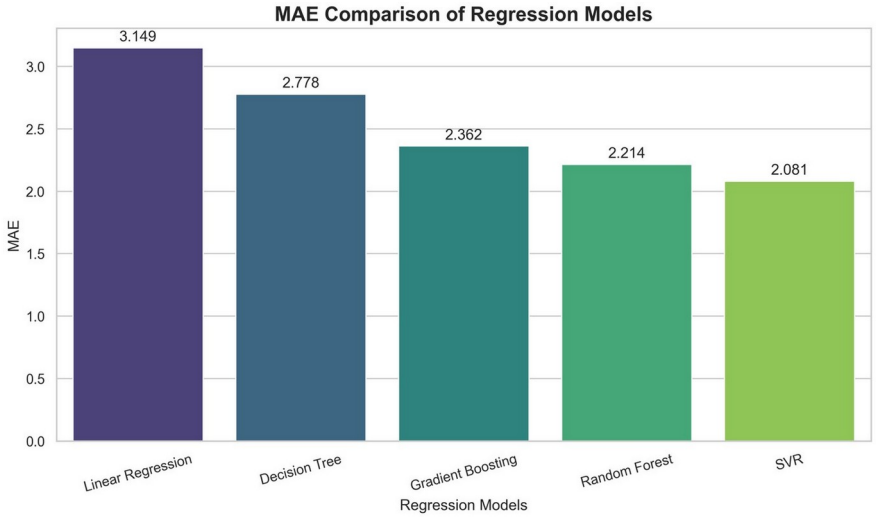


Figure 5. *MAE comparison of the five models (lower is better).*

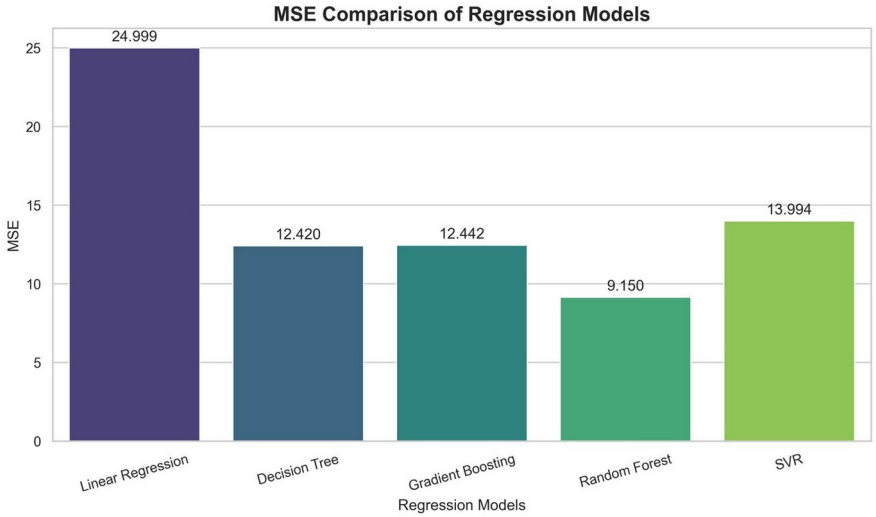


Figure 6. *MSE comparison of the five models (lower is better).*

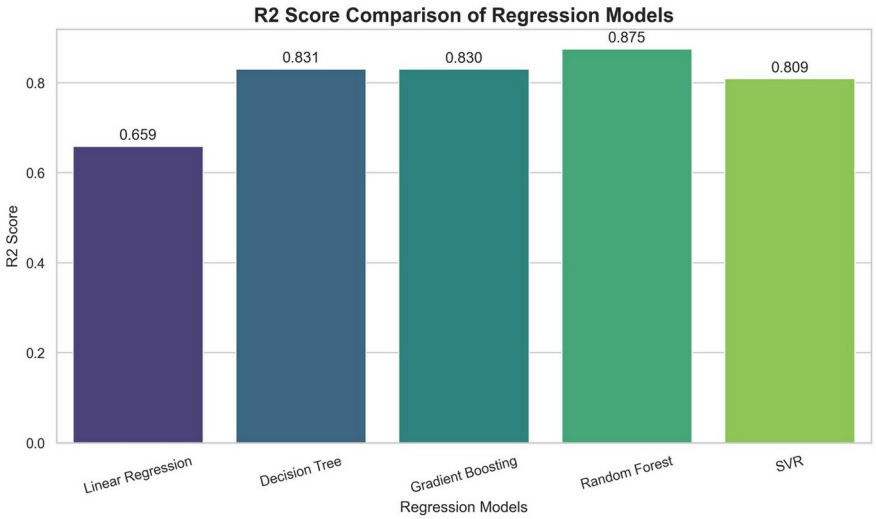


Figure 7. *R^2 score comparison of the five models (higher is better).*

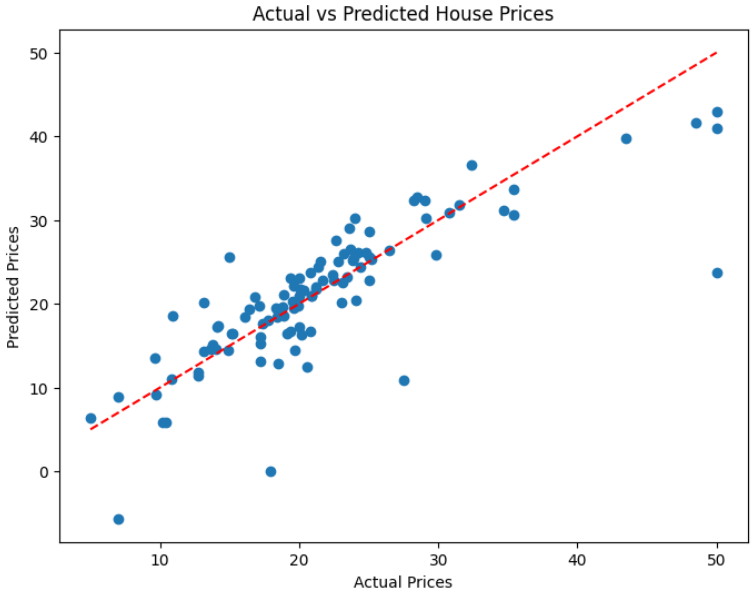


Figure 8. *Linear Regression: actual versus predicted house prices on the test set (dashed line = perfect prediction).*

Most Linear Regression predictions follow the diagonal, but expensive homes at the 50.0 ceiling are under-predicted and at least one prediction is negative, which is impossible for a price. This is consistent with its low R^2 of 0.6591.

Supplementary check: imputation fitted on training data only

Table 8. *Effect of fitting the imputers on training data only (notebook → training-only)*

Model	R^2	MSE	MAE
Linear Regression	0.6591 → 0.6593	24.9994 → 24.9834	3.1487 → 3.1476
Decision Tree	0.8306 → 0.8306	12.4201 → 12.4201	2.7776 → 2.7776
Gradient Boosting	0.8303 → 0.8304	12.4420 → 12.4347	2.3624 → 2.3641
Random Forest	0.8752 → 0.8755	9.1504 → 9.1302	2.2140 → 2.2171
SVR	0.8092 → 0.8096	13.9944 → 13.9619	2.0810 → 2.0709

This check was carried out while preparing the individual Boston project report and is not part of the original notebook.

2.7 Conclusion

The Beginner project delivered a complete regression workflow on the Boston Housing data: median and mode imputation of 120 missing values, exploratory analysis identifying RM, LSTAT and PTRATIO as the most strongly related features, an 80/20 split and a comparison of five models. **Random Forest was selected as the best overall model (MAE 2.2140, MSE 9.1504, R^2 0.8752)**, SVR achieved the lowest MAE (2.0810) and Linear Regression performed worst (R^2 0.6591).

The main limitations are the single 102-record test split without tuning or cross-validation, the capped target, and the feature B, which is derived from the proportion of Black residents in a town; the Boston Housing dataset has been removed from recent versions of scikit-learn for ethical reasons and is used here only for educational model comparison. Next steps are to use scikit-learn pipelines, cross-validation and hyperparameter tuning.

3. Intermediate-Level Project: Car Selling Price Prediction

INTERMEDIATE LEVEL
Week 2 · Project 2 of 3

Car Selling Price Prediction

3.1 Problem statement

Given the characteristics of a used vehicle (current ex-showroom price, kilometres driven, number of previous owners, age, fuel type, seller type and transmission), predict its selling price. Because the target, **Selling_Price**, is continuous, the task is a supervised regression problem. The project is documented as a notebook and saved-model deliverable; it does not include a deployed web application or API.

3.2 Dataset description

The dataset (car.csv) contains **301 listings and 9 columns** with no missing values. Two exact duplicate rows were removed, leaving **299 records**. The currency unit of the price columns is not documented, so prices are treated as relative values and all errors are in the units of Selling_Price.

Table 9. *Car dataset columns*

Column	Description
Car_Name	Name of the vehicle (98 unique names after de-duplication)
Year	Year of manufacture (2003–2018)
Selling_Price	Selling price of the vehicle (target variable)
Present_Price	Current ex-showroom price of the vehicle
Kms_Driven	Distance driven (500 – 500,000 km)
Fuel_Type	Petrol, Diesel or CNG
Seller_Type	Dealer or Individual
Transmission	Manual or Automatic
Owner	Number of previous owners (values 0, 1 and 3)

Table 10. *Summary statistics of numerical features (cleaned data, n = 299)*

Feature	Mean	Std. dev.	Min	Median	Max
Selling_Price	4.5896	4.9842	0.10	3.51	35.00
Present_Price	7.5410	8.5679	0.32	6.10	92.60
Kms_Driven	36,916.8	39,015.2	500	32,000	500,000
Year	2013.62	2.90	2003	2014	2018

The categorical features are imbalanced: Petrol 239, Diesel 58 and CNG 2; Dealer 193 and Individual 106; Manual 260 and Automatic 39; and 288 of 299 records have no previous owner.

Observation recorded in the project report

The Car_Name column shows that the file contains both cars and two-wheelers. Classifying the 98 unique names gave 198 cars and 101 two-wheelers, with very different price ranges (median Selling_Price 5.25 for cars versus 0.50 for two-wheelers). The notebook does not separate the two groups, so group effects such as the dealer–individual gap may partly reflect this mix.

3.3 Data preprocessing and feature engineering

- **Duplicates:** two duplicate records (an Ertiga and a Fortuner listing) were removed, reducing the data from 301 to 299 rows.
- **Feature engineering:** Car_Age = 2018 − Year, using 2018 (the latest year in the data) as the reference year.
- **Feature selection:** Car_Name was excluded because of its high cardinality (98 unique values for 299 rows), and Year was dropped after Car_Age was derived. The seven selected features are Present_Price, Kms_Driven, Owner, Car_Age, Fuel_Type, Seller_Type and Transmission.
- **Encoding:** Fuel_Type, Seller_Type and Transmission were one-hot encoded with OneHotEncoder(handle_unknown="ignore") inside a ColumnTransformer, giving 11 model-ready columns. The encoder was fitted on training data only.
- **Split:** 80/20 with random_state = 42, giving 239 training and 60 test records. Outliers (for example a Land Cruiser with Present_Price 92.6 and a scooter with 500,000 km) were kept, and no feature scaling was applied.

Exploratory data analysis

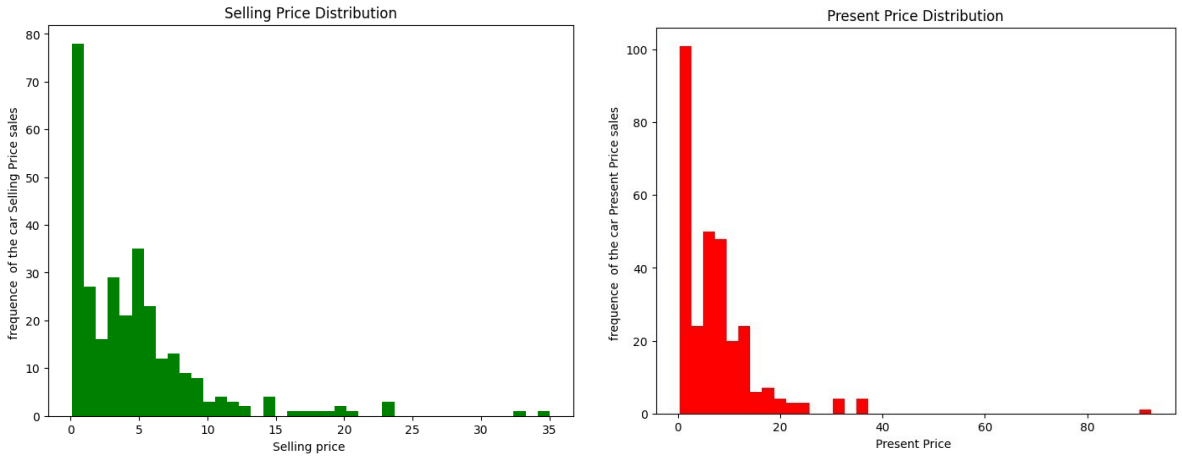


Figure 9. *Distributions of Selling_Price and Present_Price (both strongly right-skewed).*

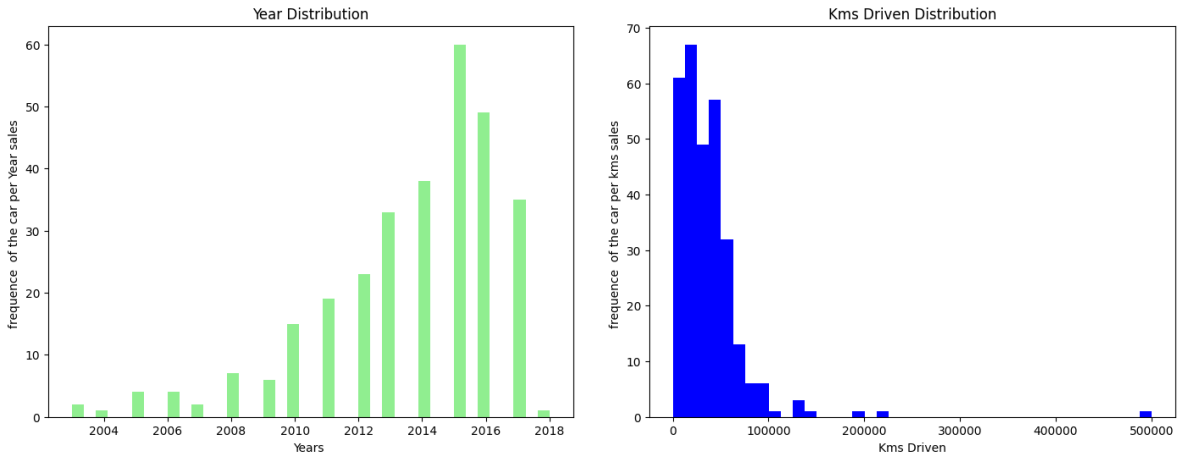


Figure 10. *Distributions of manufacturing Year and Kms_Driven.*

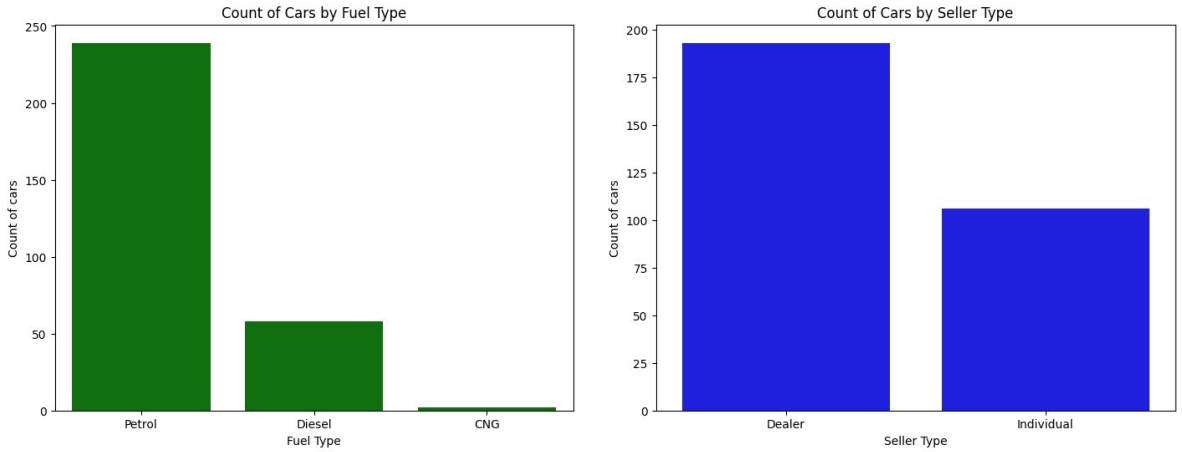


Figure 11. *Vehicle counts by Fuel_Type and Seller_Type.*

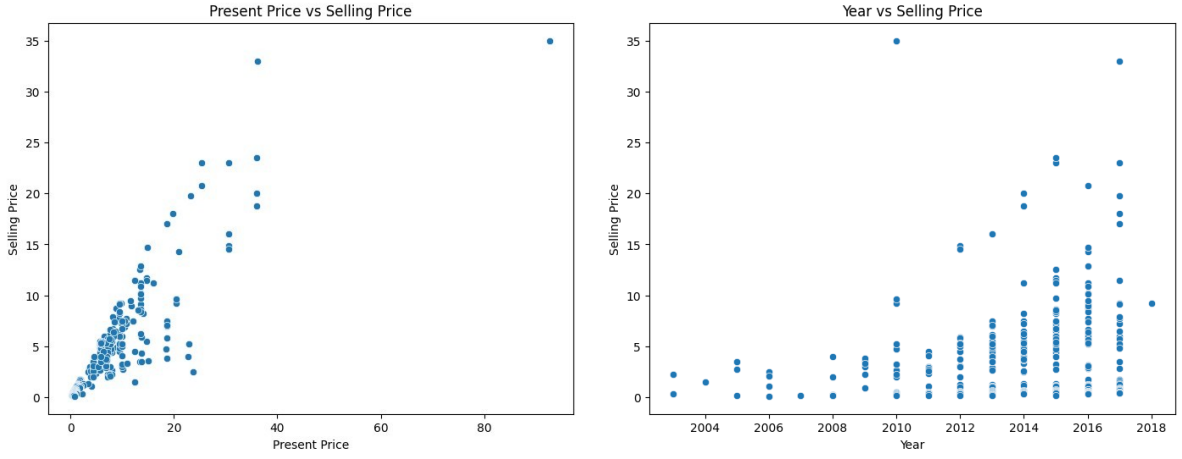


Figure 12. *Present_Price versus Selling_Price shows a clear, roughly linear upward relationship; Year versus Selling_Price is more scattered.*

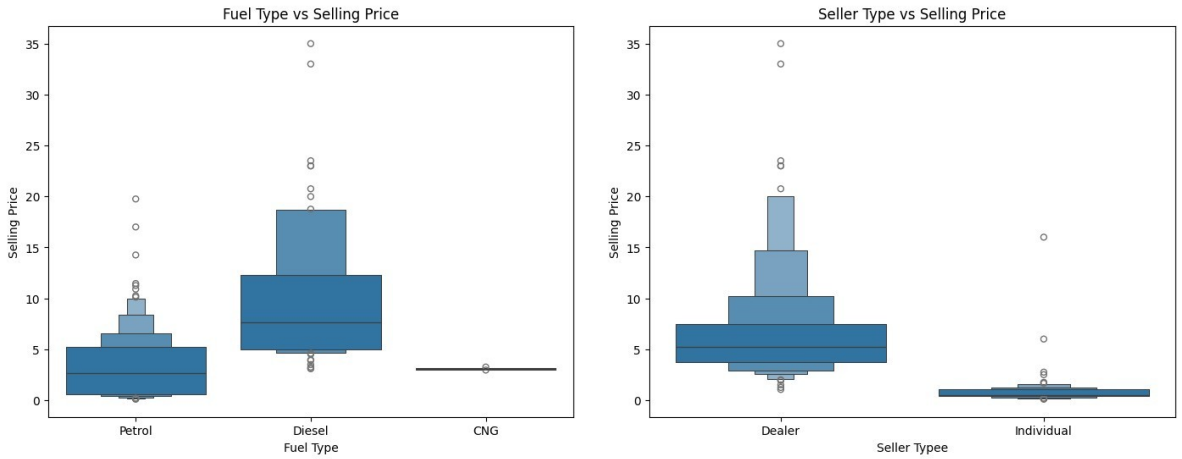


Figure 13. *Boxen plots of Selling_Price by Fuel_Type and Seller_Type.*

Median Selling_Price by group in the cleaned data is Dealer 5.25 versus Individual 0.52, Automatic 5.75 versus Manual 3.25, and Diesel 7.60, CNG 3.10 and Petrol 2.65. These differences are partly influenced by the vehicle mix, and the CNG and multi-owner groups contain very few records.



Figure 14. *Correlation heatmap of the numerical variables.*

Present_Price has by far the strongest linear association with Selling_Price (r = 0.88). Year has a weak positive correlation (0.23), Owner a weak negative one (−0.088) and Kms_Driven essentially none (+0.029). Year and Kms_Driven are moderately negatively correlated (−0.53). The notebook text describes Kms_Driven as negatively associated with Selling_Price, but the plotted coefficient is +0.029, so this report follows the plotted value.

3.4 Models implemented

Five baseline regression models were trained on the 239 encoded training records, with random_state = 42 for the tree-based models and all other hyperparameters at scikit-learn defaults: Linear Regression, Decision Tree Regressor, Random Forest Regressor, Gradient Boosting Regressor and Support Vector Regressor (default settings, without feature scaling).

Random Forest and Gradient Boosting were then tuned with **RandomizedSearchCV**, each wrapped in a Pipeline with the preprocessor so that encoding is refitted inside every cross-validation fold. For each model, 50 random combinations were sampled and evaluated with five-fold cross-validation on the training data (250 fits per model), using negative RMSE as the scoring objective.

Table 11. *Best hyperparameters found by RandomizedSearchCV*

Hyperparameter	Random Forest	Gradient Boosting
n_estimators	100	300
max_depth	20	4
min_samples_split	2	5
min_samples_leaf	2	4
max_features	1.0	–
learning_rate	–	0.1
subsample	–	0.8
Best cross-validation RMSE	1.5436	1.3817

Both tuned models have a cross-validation RMSE (1.54 and 1.38) lower than their RMSE on the test split (3.27 and 3.07). The project report attributes this gap to the small dataset: the 60-record test split may contain a few high-priced vehicles that are harder to predict.

3.5 Model comparison and actual results

Table 12. *Car test-set results for all seven models (sorted by R^2)*

Model	MAE	MSE	RMSE	R^2
Gradient Boosting (Baseline)	1.1349	6.3667	2.5232	0.7530
Linear Regression	1.4729	6.3708	2.5240	0.7528
Decision Tree	1.3273	8.4855	2.9130	0.6708
Gradient Boosting (Tuned)	1.3016	9.4343	3.0715	0.6339
Random Forest (Tuned)	1.3851	10.6659	3.2659	0.5862
Random Forest (Baseline)	1.5270	13.7505	3.7082	0.4665
SVR (Baseline)	3.1295	28.3205	5.3217	−0.0988

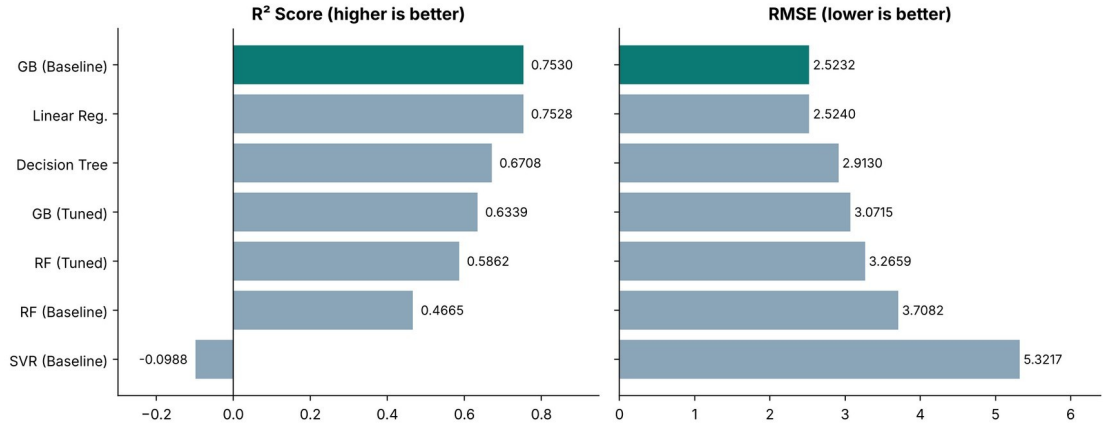


Figure 15. *R^2 and RMSE of all seven models on the test set.*

The baseline Gradient Boosting Regressor achieved the highest R^2 (0.7530), the lowest RMSE (2.5232) and the lowest MAE (1.1349). Its advantage over Linear Regression is very small on R^2 and RMSE (0.7530 versus 0.7528 and 2.5232 versus 2.5240); the clearer difference is in MAE (1.1349

versus 1.4729). With only 60 test records, the two models should be regarded as performing similarly on squared-error metrics.

Tuning improved Random Forest (R^2 from 0.4665 to 0.5862) but reduced Gradient Boosting's test performance (R^2 from 0.7530 to 0.6339), so the tuned models did not outperform the baseline Gradient Boosting model on the test split. SVR performed worst, with a negative R^2 (−0.0988); because it was trained without feature scaling, this should not be read as a fair assessment of the algorithm.

3.6 Feature importance

Feature importance was extracted from the baseline Gradient Boosting model fitted on the encoded training data.

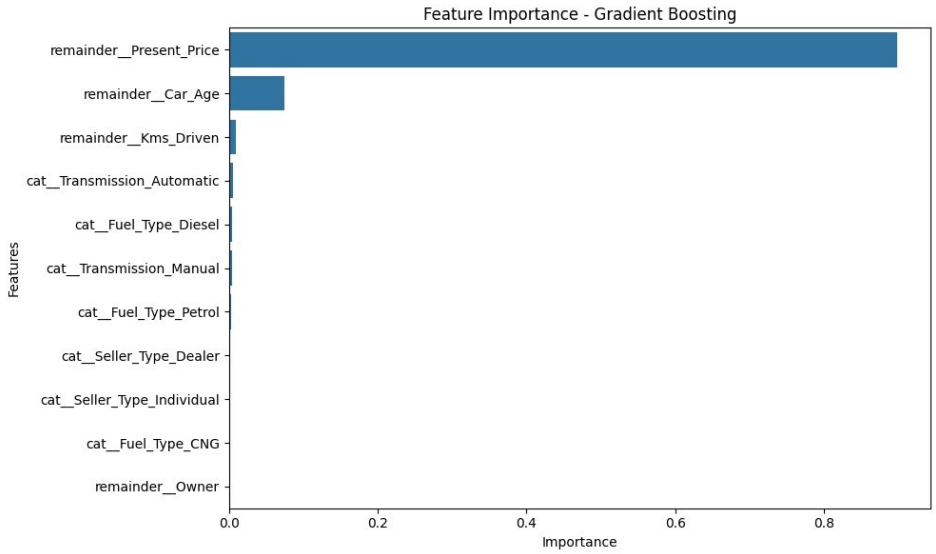


Figure 16. *Feature importance of the baseline Gradient Boosting model.*

Table 13. *Feature importances (baseline Gradient Boosting)*

Feature	Importance
Present_Price	0.898545
Car_Age	0.074910
Kms_Driven	0.009688
Transmission_Automatic	0.005286
Fuel_Type_Diesel	0.004270
Transmission_Manual	0.003892
Fuel_Type_Petrol	0.003307
Seller_Type_Dealer	0.000071
Seller_Type_Individual	0.000025
Fuel_Type_CNG	0.000004
Owner	0.000002

Present_Price dominates (about 89.85%), followed by Car_Age (about 7.49%) and Kms_Driven (about 0.97%). This is consistent with the exploratory analysis: the current price of a vehicle is a strong signal of its resale value, and age captures depreciation.

3.7 Conclusion

The Intermediate project produced a complete regression workflow for predicting used-vehicle selling prices, from cleaning and the Car_Age feature through five baselines, tuned ensembles, evaluation and feature-importance analysis. **The baseline Gradient Boosting Regressor performed best (MAE 1.1349, RMSE 2.5232, R^2 0.7530)**, and the final preprocessor and model were saved with Joblib as best_car_price_model.pkl; the project report records that reloading this file reproduced R^2 = 0.7530 and MAE = 1.1349 on the test split.

The limitations are the small sample (299 records), the use of test-set results during model comparison (so reported performance is exploratory), SVR trained without scaling, the mixture of cars and two-wheelers, and very small CNG and multi-owner groups. Suggested next steps are to select models by cross-validation, model cars and two-wheelers separately, and wrap the saved model in a simple application.

4. Advanced-Level Project: ReviewMind AI

ADVANCED LEVEL
Weeks 3–4 · Project 3 of 3

ReviewMind AI — Universal Customer Feedback Intelligence Platform

4.1 Problem statement

Organisations receive large volumes of unstructured customer feedback that is slow and inconsistent to review by hand. The problem addressed was to design a single platform that (i) accepts arbitrary tabular feedback files, (ii) identifies the usable text and label columns, (iii) applies reliable sentiment, emotion and topic analysis, and (iv) presents the outcome through visualisations and plain-language answers, while remaining runnable locally without paid external language-model APIs.

4.2 Project objectives

- Build a sentiment-classification workflow on the IMDb review dataset, from data inspection and cleaning to evaluation on a held-out test set.
- Compare a classical TF-IDF + Logistic Regression model with a fine-tuned DistilBERT transformer on identical test data.
- Add emotion detection with a pretrained DistilRoBERTa model and topic discovery with TF-IDF and NMF.
- Develop a modular backend that profiles data, detects text and label columns, preprocesses text and orchestrates the selected analyses.
- Deliver an interactive Streamlit application with upload, analysis, visualisation, export and a local rule-based assistant.
- Verify the backend through integration tests covering normal, unlabeled, messy and unusual inputs.

Scope: binary (positive/negative) sentiment, English-language emotion classification, unsupervised topic discovery and exploratory visualisation. Quantitative evaluation was performed on the IMDb dataset only; multilingual support, cloud deployment and production-scale benchmarking are outside the present scope.

4.3 System architecture and workflow

The project has two connected parts. The **model-development notebook** (ReviewMind_AI.ipynb) trains and evaluates the sentiment models. The **application and backend** (app/app.py and the src/ modules) package those models, together with emotion detection and topic modelling, into a tool that works on any uploaded feedback file.

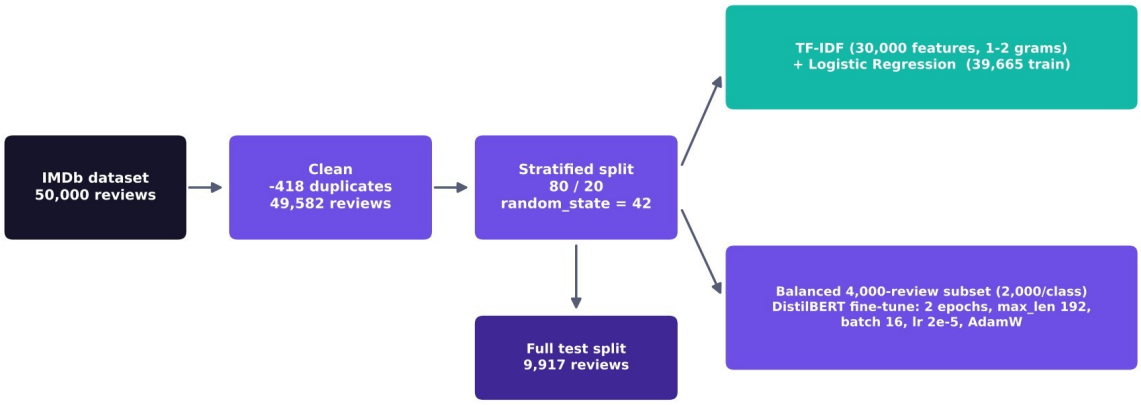


Figure 17. *Model-development pipeline for the sentiment models (from the ReviewMind AI project report).*

The application workflow has six steps: (1) the user uploads a CSV or Excel file or selects the demo dataset; (2) the application profiles the data (rows, columns, missing values, duplicates, empty rows, data types and blank strings); (3) candidate feedback-text and label columns are detected and the user confirms the feedback column and, optionally, a label column; (4) the user chooses Automatic or Custom mode and a sentiment model; (5) rows with missing or blank feedback are excluded and predictions, confidences, emotions and topics are appended to a results table; and (6) results are explored in the Visualization Center, exported as CSV or PDF, and queried through Ask ReviewMind.

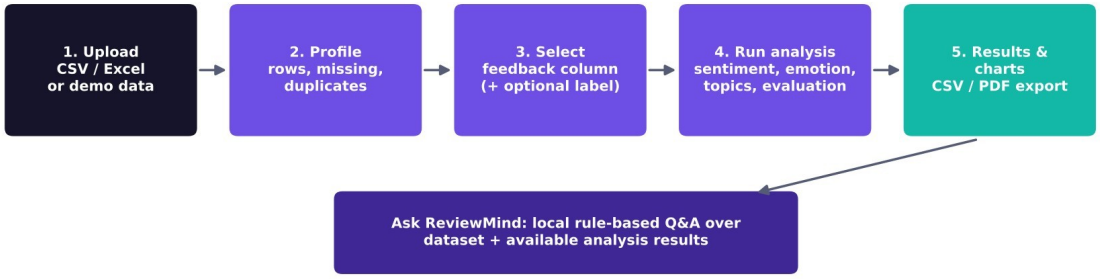


Figure 18. *Processing workflow of the ReviewMind AI application (from the ReviewMind AI project report).*

The backend is modular: data_loader (CSV/Excel loading and workbook inspection), data_profiler (profiling and text/label column detection), preprocessing (text cleaning), sentiment, emotion, topics, evaluation, analysis_engine (run_analysis and run_automatic_analysis), assistant_engine, visualization and exporter. Separating these concerns allows each function to be tested independently.

Table 14. *Technology stack (versions printed by the notebooks on the development machine)*

Category	Tools
Language	Python 3.11.9
Application framework	Streamlit
Data processing	Pandas 3.1.1, NumPy 2.4.6
Machine learning	Scikit-learn 1.9.1 (TfidfVectorizer, LogisticRegression, NMF, metrics)
Deep learning / NLP	PyTorch 2.15.0 (CPU build), Hugging Face Transformers 5.22.2, Accelerate 1.15.0

Category	Tools
Pretrained models	distilbert/distilbert-base-uncased (fine-tuned); j-hartmann/emotion-english-distilroberta-base
Visualisation	Matplotlib, Seaborn, Streamlit charts
Model storage	Joblib (.pkl) and Hugging Face save_pretrained artifacts
Development and version control	Jupyter Notebook, Visual Studio Code, Git and GitHub

4.4 Dataset and preprocessing

The primary dataset is the IMDb Movie Reviews dataset (Zenodo record 7928582), with columns review (free text with some HTML line-break markup) and sentiment (positive or negative). The raw file is perfectly balanced with 25,000 reviews per class, has no missing values and includes 418 exact duplicate rows.

Table 15. *Dataset summary*

Property	Value
Raw reviews	50,000 (25,000 positive / 25,000 negative)
Missing values	0 in both columns
Duplicate rows	418
Reviews after cleaning	49,582 (24,884 positive / 24,698 negative)
Training split (80%)	39,665 reviews (19,907 positive, 19,758 negative)
Test split (20%)	9,917 reviews (4,977 positive, 4,940 negative)

Duplicates were dropped and the index reset, leaving 49,582 reviews. The HTML markup (
) was replaced by a space with a regular expression so that markup tokens do not become model features, and labels were mapped to integers (negative = 0, positive = 1). The data was split 80/20 with train_test_split(test_size=0.20, random_state=42, stratify=y); the TF-IDF vectoriser was fitted on training text only, preventing vocabulary leakage.

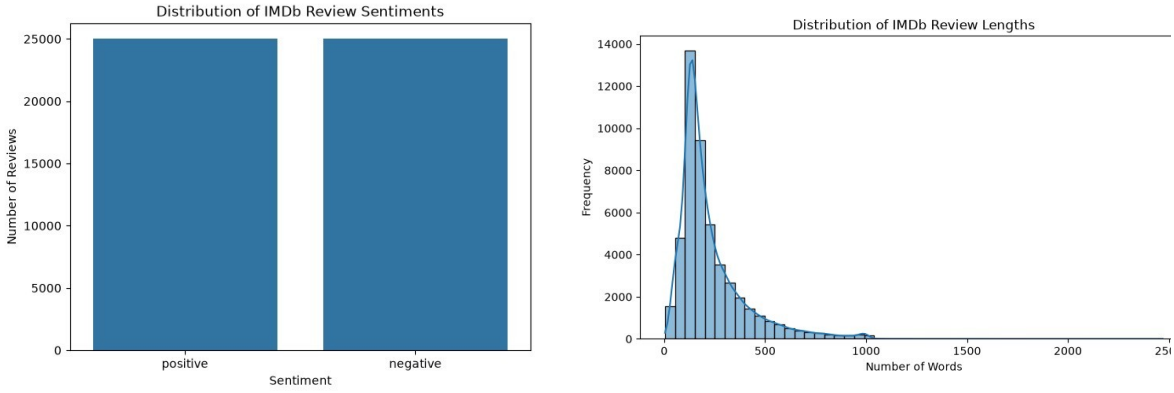


Figure 19. *Class distribution of the raw IMDb dataset and distribution of review lengths in words.*

Review length is strongly right-skewed (mean 231.35 words, median 173, 90th percentile 452, maximum 2,470). This matters for transformer models, which accept a fixed maximum length: DistilBERT used 192 tokens, so longer reviews were truncated, whereas the TF-IDF model reads the full text.

4.5 Sentiment analysis

Model 1: TF-IDF + Logistic Regression

TF-IDF represents each review as a sparse vector weighted by how often terms occur in the review and how distinctive they are across the corpus; Logistic Regression then learns a linear decision boundary and outputs class probabilities, which the application reports as confidence scores. The approach is fast, needs no GPU and keeps word and phrase weights relatively interpretable. The vectoriser and model are saved together as models/tfidf_logistic_regression.pkl.

Table 16. *TF-IDF and Logistic Regression configuration*

Parameter	Value
max_features	30,000
ngram_range	(1, 2): unigrams and bigrams
min_df / max_df	2 / 0.95
Training / test matrix shape	39,665 × 30,000 / 9,917 × 30,000
Classifier	LogisticRegression(max_iter=1000, random_state=42)

Model 2: fine-tuned DistilBERT

DistilBERT is a compact transformer distilled from BERT that produces contextual token representations, allowing the classifier to account for word order and surrounding context. The distilbert/distilbert-base-uncased checkpoint was loaded with a two-class head and fine-tuned with a custom PyTorch loop. Because no GPU was available, training used a balanced subset of 4,000 reviews (2,000 per class), which took about 17.8 minutes per epoch and roughly 35.7 minutes in total.

Table 17. *DistilBERT fine-tuning configuration*

Parameter	Value
Device	CPU (CUDA not available)
Training data	4,000 reviews (2,000 per class)
Maximum sequence length	192 tokens
Batch size	16 (250 steps per epoch)
Epochs / total steps	2 / 500
Optimiser and learning rate	AdamW, 2e-5
Scheduler	Linear schedule with 10% warm-up (50 steps)
Gradient clipping	Maximum norm 1.0
Saved artifact	models/distilbert_imdb_sentiment/ (config, safetensors weights, tokenizer)

4.6 Emotion detection

Emotion detection uses the pretrained **j-hartmann/emotion-english-distilroberta-base** model through a Hugging Face text-classification pipeline on CPU, with truncation enabled. It was **not fine-tuned or retrained** in this project. In the recorded runs it returned the labels joy, sadness, surprise, disgust, fear and neutral, and the backend wrapper returns an empty list for empty, None or blank input instead of raising an error.

Table 18. *Emotion predictions on five hand-written sentences (recorded output)*

Sentence (abridged)	Predicted emotion	Confidence
“This movie was absolutely fantastic. I loved every moment!”	joy	0.9716
“I am extremely disappointed with this terrible movie.”	fear	0.4298
“The movie made me feel very sad and emotional.”	sadness	0.9864
“I was surprised by the unexpected ending.”	surprise	0.9831
“It was an ordinary movie with nothing particularly special.”	surprise	0.6098

Clear emotional statements receive high-confidence labels, whereas the ambiguous second and fifth sentences receive low or moderate confidence and arguably unexpected labels. This is why confidence scores are shown to users and why emotion output should be reviewed rather than treated as fact. The emotion component was demonstrated through integration tests and sample outputs and was not quantitatively evaluated.

4.7 Topic modelling

Topics are discovered without labels using TF-IDF features and Non-negative Matrix Factorization (NMF), which decomposes the document-term matrix into additive topics characterised by their highest-weighted keywords. The recorded run used TfidfVectorizer(max_features=5000, min_df=2, max_df=0.95, stop_words="english") and NMF(n_components=5, init="nndsvda", max_iter=400, random_state=42); each document is assigned to its dominant topic.

Table 19. *Topics discovered by NMF on the first 100 IMDb reviews (recorded output)*

Topic	Top keywords
1	br, movie, say, just, young, great, characters, scene
2	movie, like, actors, bad, good, watch, just, thing
3	film, just, version, story, films, did, lot, far
4	war, br, film, hollywood, mountain, romantic, beautiful, story
5	movies, seen, japanese, horror, ve, tv, series, water

The keywords describe recurring vocabulary rather than verified business categories, so the topics are broad exploratory themes. The residual token “br” in topics 1 and 4 comes from HTML markup because this call passed raw review text to discover_topics; cleaning text with clean_text first avoids this.

4.8 Ask ReviewMind assistant

Ask ReviewMind is a floating chat assistant that answers a defined set of question categories — Dataset Overview, Sentiment Analysis, Emotion Analysis, Topics and Keywords, Customer Complaints and Business Insights — from the uploaded dataset and the available analysis results. It is **rule-based and runs locally**, so it does not depend on a paid external language-model API. Responses can include small charts (limited to six in the code), and the assistant state is reset when the dataset changes so that answers never refer to a previous file. It is not a general conversational model.

4.9 Interactive dashboard and visualisations

The Streamlit application (app/app.py, styled by app/style.css) has a sidebar for choosing the data source and workspace, and five workspaces:

- **Overview:** dataset health metrics (rows, columns, missing values, duplicate rows, empty rows) and an expandable schema and data-quality table.
- **Analyze:** feedback-column selection with a recommended column and usable-row coverage, an optional ground-truth label, Automatic or Custom mode, model choice and a Run analysis button; results include original, analysed and excluded row counts, evaluation metrics when labels exist, a results table and a CSV download.
- **Visualizations:** a Custom Graph Builder plus charts that appear only when the data allows them (sentiment donut, emotion distribution, topic frequency, rating distribution, keyword frequency, date trend, numeric distribution and correlation heatmap), each with a PNG download.
- **Exports:** the analysis CSV and a generated PDF report.
- **Key Terms:** plain-language definitions of sentiment, emotions, confidence score, TF-IDF, Logistic Regression, DistilBERT, topic modelling and evaluation metrics.

The interface uses a violet and teal colour scheme with a dark sidebar, preserves widget state when moving between workspaces, keeps the original uploaded data unchanged and states its design principles: no paid APIs, no arbitrary code execution and original data preserved. Running all four analyses on the first 100 IMDb reviews produced the columns predicted_sentiment, sentiment_confidence, predicted_emotion, emotion_confidence and discovered_topic; their recorded distributions are shown below.

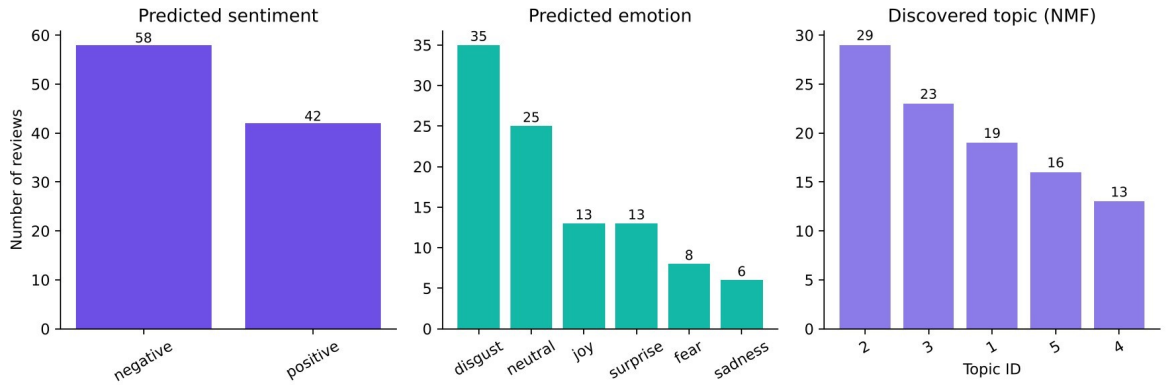


Figure 20. *Predicted sentiment, emotion and topic distributions for the first 100 IMDb reviews (recorded integration-test output).*

> **Note on screenshots**
>
> The supplied ReviewMind AI materials contain workflow diagrams and result charts but no screenshots of the running Streamlit interface, so none are reproduced here.

The integrated evaluation on these 100 reviews reported 96% accuracy (confusion matrix [[56, 2], [2, 40]]). This should not be read as a performance estimate: the sample is small and consists of the first rows of the raw dataset, which may overlap with the Logistic Regression training data. The held-out results in Section 4.10 are the valid measure of model quality.

4.10 Evaluation results

Both sentiment models were evaluated on the same held-out test split of 9,917 reviews that neither model saw during training. Metrics use the positive class as the reference.

TF-IDF + Logistic Regression

Table 20. *Logistic Regression performance on the full test split (n = 9,917)*

Metric	Result
Accuracy	0.9052 (90.52%)
Precision	0.8945
Recall	0.9196
F1-score	0.9069
ROC-AUC	0.9663

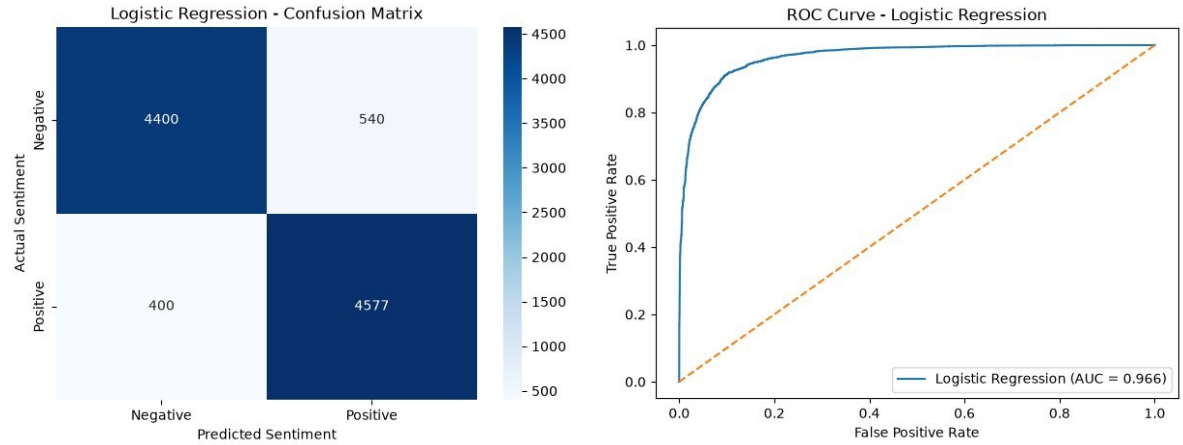


Figure 21. *Logistic Regression confusion matrix (4,400 / 540 / 400 / 4,577) and ROC curve (AUC = 0.966) on the 9,917-review test split.*

The model makes 940 incorrect predictions (540 false positives and 400 false negatives), an error rate of about 9.5%.

Fine-tuned DistilBERT

Table 21. *DistilBERT training history*

Epoch	Average training loss	Time (minutes)
1	0.4410	17.83
2	0.2297	17.88

Table 22. *DistilBERT evaluation: 1,000-review subset versus full test split*

Metric	Subset (n = 1,000)	Full test split (n = 9,917)
Accuracy	0.8700	0.8756
Precision	0.8887	0.8947
Recall	0.8460	0.8523
F1-score	0.8668	0.8730
ROC-AUC	0.9517	0.9501

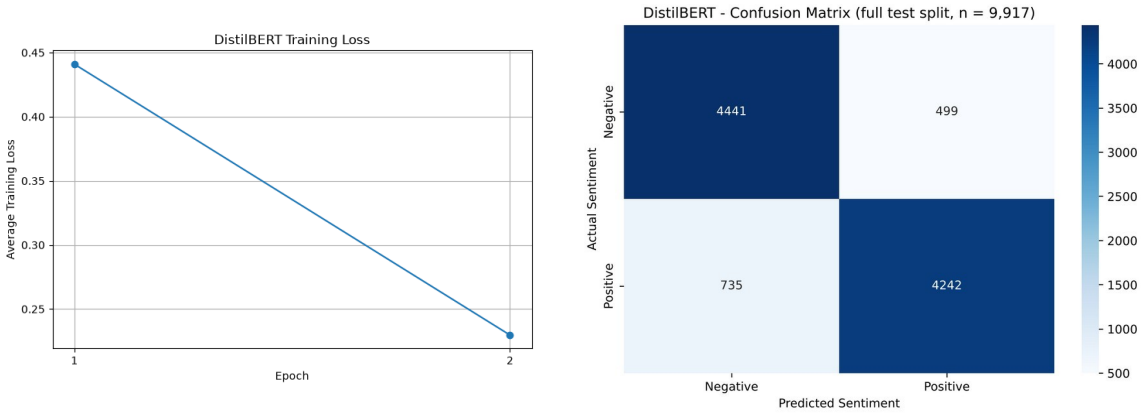


Figure 22. *DistilBERT average training loss by epoch and confusion matrix on the full test split (4,441 / 499 / 735 / 4,242).*

DistilBERT makes 1,234 errors (499 false positives and 735 false negatives), about 12.4%. It misses more positive reviews than it wrongly flags, which explains its lower recall (0.8523) relative to its precision (0.8947). Falling training loss alone does not establish generalisation, so the held-out metrics are the relevant evidence.

Model comparison

Table 23. *Logistic Regression versus DistilBERT on the same 9,917-review test split*

Metric	Logistic Regression	DistilBERT	Difference (LR − DB)
Accuracy	0.9052	0.8756	+0.0296
Precision	0.8945	0.8947	−0.0002
Recall	0.9196	0.8523	+0.0673
F1-score	0.9069	0.8730	+0.0339
ROC-AUC	0.9663	0.9501	+0.0162

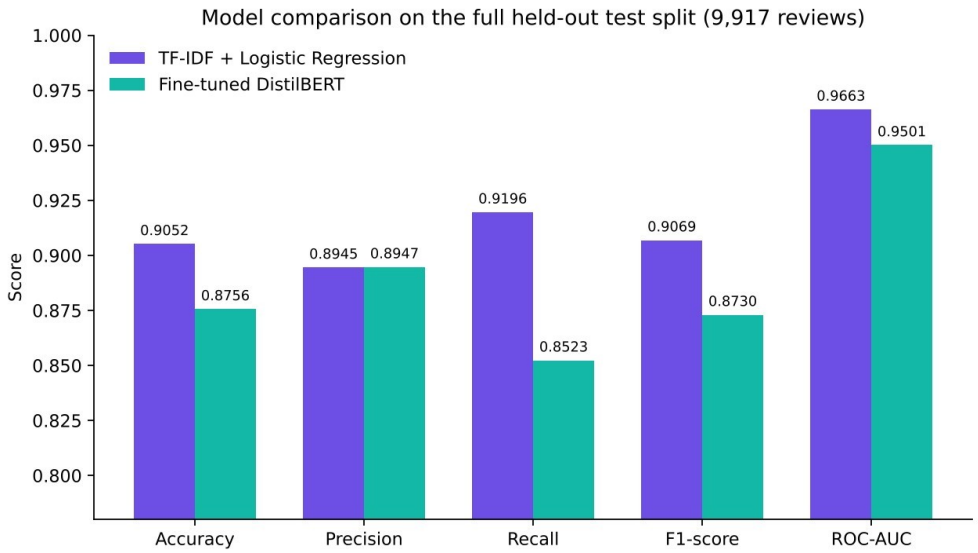


Figure 23. *Side-by-side metrics for both sentiment models on the full held-out test split.*

How to read this comparison

Logistic Regression was the stronger model on accuracy, recall, F1-score and ROC-AUC in this experiment, while precision was effectively tied. The result should not be generalised: Logistic Regression was trained on all 39,665 training reviews whereas DistilBERT saw only 4,000 (about 10%), for two epochs, with inputs truncated to 192 tokens. The comparison reflects this experimental configuration and is not a general ranking of the architectures.

Error analysis

The notebook displays five false positives and five false negatives from the Logistic Regression model. Several begin with wording that suggests mixed sentiment, for example a negative review opening “Despite being a huge fan of Fred Astaire and G…” that was predicted positive. A bag-of-words model detects strongly polarised terms but cannot reliably resolve such contrasts. This observation comes from the few displayed examples and was not quantified across all 940 errors.

Backend integration testing

The notebook ReviewMind_Backend_Testing.ipynb is an integration smoke test of the public backend functions; it does not retrain models. Its recorded report lists **10 of 10 checks passed**.

Table 24. *Backend integration test report (recorded output)*

Check	Result
Dataset loading and profiling	PASS
Sentiment + emotion + topics + evaluation in one run	PASS
Prediction output consistency (expected columns, row count)	PASS
Custom sentiment-only analysis	PASS
Custom emotion-only analysis	PASS
Default analysis (sentiment + topics)	PASS
Unlabeled dataset (evaluation skipped)	PASS
Missing and blank text (30 rows in, 27 analysed)	PASS
CSV and Excel loading (20 × 2 each)	PASS
Different feedback dataset (20 product comments)	PASS

Further checks confirmed that invalid inputs (an unknown text column, an empty dataset, an unsupported analysis name and a column with no usable text) raise informative errors, and that column detection correctly identified review and sentiment in the IMDb data and customer_feedback in the product-feedback sample. These tests are evidence of correct behaviour, not of model quality.

4.11 Deployment and limitations

Deployment

The platform runs **locally**: clone the repository, create a virtual environment, run pip install -r requirements.txt, and start the application with streamlit run app/app.py. The code is hosted on GitHub (github.com/samarthjain-ai/ShadowFox-ML-Internship). The large DistilBERT weights cannot be stored in the repository and must be supplied separately before that model is selected; the TF-IDF + Logistic Regression model is available in the repository layout. No cloud deployment or hosted demonstration was part of this project.

Limitations

- Training and quantitative evaluation used only IMDb movie reviews; predictions may be less reliable for other domains, and the backend was smoke-tested on only a small product-feedback sample.
- Sentiment is binary, so neutral and mixed opinions are forced into positive or negative.
- DistilBERT was trained on a 4,000-review subset for two epochs with truncated inputs, so its measured performance is not a ceiling for the architecture.
- Emotion classification can misinterpret ambiguous text and was not quantitatively evaluated; topics are exploratory keyword groups.
- Ask ReviewMind is rule-based and limited to defined question categories; some visualisations require particular column types such as dates or numeric ratings.
- Predictions are analytical estimates, not verified facts about individual customers; uploaded data may contain sensitive information, and models may reflect biases of the IMDb training data.

4.12 Conclusion

ReviewMind AI demonstrates a complete machine-learning workflow, from data inspection and cleaning through model training, evaluation and persistence to integration in an interactive application. **On the held-out IMDb test split of 9,917 reviews, TF-IDF with Logistic Regression reached 90.52% accuracy and a 0.9663 ROC-AUC, outperforming a CPU-trained DistilBERT model that reached 87.56% accuracy and a 0.9501 ROC-AUC** under reduced training conditions. The platform also provides emotion detection, topic discovery, visual analytics, exports and a local rule-based assistant, and its backend passed all ten recorded integration checks.

The most important lessons were methodological: compare models on identical test data, state the training conditions behind each result, and treat integration-test output as evidence of correct behaviour rather than model quality. The single-domain training data defines the clearest next step: evaluation and adaptation on real customer-feedback datasets.

5. Overall Learning Outcomes

The three projects together exercised the full machine-learning lifecycle. The outcomes below are limited to what the project materials demonstrate.

5.1 Machine learning concepts

- Supervised regression and classification, and the difference between predicting a continuous target (house and car prices) and a class label (sentiment).
- Linear models, decision trees, ensemble methods (Random Forest and Gradient Boosting), Support Vector Regression, TF-IDF with Logistic Regression, and a fine-tuned transformer (DistilBERT).
- Unsupervised topic discovery with NMF and the use of a pretrained emotion-classification model without retraining it.
- Overfitting and generalisation: falling training loss in DistilBERT is not evidence of generalisation, and tuned models in the Car project did not beat a baseline on the test split.

5.2 Data analysis

- Data quality checks: missing values (120 in the Boston data), duplicates (2 in the Car data, 418 in the IMDb data), capped targets, outliers and imbalanced categories.
- Exploratory analysis with distributions, correlation heatmaps, scatter plots and box plots, and the habit of reading plotted values rather than assumed ones (the Kms_Driven correlation in the Car project).
- Feature engineering and encoding (Car_Age, one-hot encoding) and text cleaning for NLP (HTML markup removal, TF-IDF n-grams).

5.3 Model evaluation

- Choosing metrics to fit the task: MAE, MSE, RMSE and R^2 for regression; accuracy, precision, recall, F1-score, ROC-AUC and confusion matrices for classification.
- Comparing models on identical held-out data, and stating the training conditions behind each result so that comparisons are not over-interpreted.
- Recognising the limits of small test sets, single splits, exploratory use of the test set and preprocessing before the split, and quantifying their effect where possible (Boston supplementary check).

5.4 Python development

- Scripting analysis in Jupyter notebooks with Pandas, NumPy, Matplotlib, Seaborn and scikit-learn, including Pipeline, ColumnTransformer and RandomizedSearchCV.
- Writing a custom PyTorch training loop for DistilBERT with AdamW, a linear warm-up scheduler and gradient clipping.
- Structuring code into small, separately testable modules (data loading, profiling, preprocessing, sentiment, emotion, topics, evaluation, orchestration, visualisation and export).

5.5 Application development

- Building a Streamlit application with upload, column detection, automatic and custom analysis modes, interactive charts, CSV and PDF export and a local rule-based assistant.

- Handling messy input: blank and missing text, unlabeled data, alternative column names, Excel files and clear error messages; and writing integration tests (10 of 10 recorded checks passed).

5.6 Deployment and GitHub

- Persisting trained artifacts: a Joblib bundle of preprocessor and model for the Car project, and a Joblib TF-IDF + Logistic Regression artifact plus Hugging Face save_pretrained files for ReviewMind AI.
- Documenting how to run the projects (dependencies, run commands, repository layout) and using Git and GitHub for version control and project hosting.
- Understanding practical deployment constraints, such as large model weights that cannot be stored in a repository. A hosted cloud deployment was not part of this internship.

6. Challenges Faced and Solutions

Table 25. *Challenges and how they were addressed*

Project	Challenge	Solution or mitigation
Boston	Missing values in six columns (3.95% each)	Median imputation for numeric columns and mode for binary CHAS; completeness verified afterwards.
Boston	Possible outliers in MEDV (40 flagged by the IQR rule) and a capped target (16 houses at 50.0)	Outliers inspected and retained; the cap was identified in the EDA and documented as a caution for expensive homes.
Boston	Scale-sensitive SVR and correlated predictors (RAD and TAX, r = 0.91)	SVR trained in a pipeline with StandardScaler; a feature-to-feature heatmap guided interpretation and tree-based models were included.
Boston	Imputation applied before the train/test split	Documented transparently and quantified in a supplementary check; the effect on the metrics was negligible.
Car	High-cardinality Car_Name (98 names for 299 rows) and Year as a calendar value	Car_Name excluded; Car_Age = 2018 − Year engineered and Year removed.
Car	Duplicates, skewed distributions and extreme values	Two duplicates removed; skewed values kept since tree-based models are not sensitive to feature scale.
Car	Categorical features and tuning inside cross-validation	One-hot encoding in a ColumnTransformer; the preprocessor placed inside a Pipeline so each fold is encoded separately.
Car	Tuned models did not beat the baseline on the test split	Reported as recorded; baseline Gradient Boosting retained; role of the small dataset and single split explained.
ReviewMind AI	No GPU available for transformer training	DistilBERT fine-tuned on a balanced 4,000-review subset for two epochs with a 192-token limit (about 35.7 minutes in total).
ReviewMind AI	Noisy text (HTML markup, 418 duplicates) and long reviews versus a fixed transformer input length	Duplicates dropped and markup replaced; review-length percentiles analysed and truncation acknowledged as a limitation.
ReviewMind AI	Fair comparison of two differently trained models	Both evaluated on the identical 9,917-review test split with unequal training conditions stated explicitly.
ReviewMind AI	Emotion model download interrupted and disk space reported insufficient	Download timeouts raised, a single download worker used, and the model stored locally in models/emotion_english_distilroberta/.
ReviewMind AI	Large DistilBERT weights cannot be stored in the GitHub repository	Repository documents that the weights must be supplied separately before that model is used.
ReviewMind AI	Uploaded data with missing, blank or unlabeled text; Streamlit state lost between pages	Blank rows excluded but accounted for (30 in, 27 analysed); evaluation skipped with a clear message; widget values persisted across workspaces and the loaded dataset cached.

7. Future Improvements

7.1 Project-specific improvements

Table 26. *Planned next steps for each project (from the project reports)*

Project	Planned next steps
Boston House Price Prediction	Fit all preprocessing inside a scikit-learn Pipeline; use cross-validation and hyperparameter tuning (GridSearchCV or RandomizedSearchCV); report RMSE, cross-validated scores and feature importance; evaluate feature selection and the effect of removing sensitive features such as B; try a log-transformed target; test on more recent datasets; save the final model and wrap it in a simple interface.
Car Selling Price Prediction	Select models by cross-validation and keep the test set for one final evaluation; scale features for SVR and compare again; model cars and two-wheelers separately or add a vehicle-type feature; try log(Selling_Price); use repeated cross-validation for confidence intervals; collect a larger and more recent dataset; wrap the saved model in a web form or API.
ReviewMind AI	Evaluate and adapt models on real customer-feedback datasets (domain adaptation); fine-tune DistilBERT on the full training split with a validation set, longer sequences and more epochs on a GPU, then repeat the comparison under equal conditions; add neutral or fine-grained sentiment and multilingual support; clean markup before topic modelling and add coherence measures; expand the assistant, reporting and visualisation options; optimise models for cloud deployment and publish a hosted demonstration; extend automated tests.

7.2 Cross-cutting improvements

- Adopt a consistent, leakage-safe evaluation protocol across projects: preprocessing inside pipelines, model selection by cross-validation and a single final test evaluation.
- Standardise packaging so that every project ends with a saved model, a documented run command and a simple prediction interface.
- Add real screenshots of the running application to the documentation.

8. Overall Internship Conclusion

The ShadowFox Machine Learning Engineer Internship (1–30 November 2026) was completed through three projects of increasing depth. The **Beginner** project established the core regression workflow on 506 Boston housing records and selected Random Forest (R^2 0.8752). The **Intermediate** project added feature engineering, hyperparameter tuning, feature-importance analysis and model persistence on 299 used-vehicle records, where the baseline Gradient Boosting Regressor performed best (R^2 0.7530) and Present_Price accounted for about 89.85% of its importance. The **Advanced** project combined classical and transformer NLP models, emotion detection and topic modelling in a tested, modular Streamlit application, where TF-IDF with Logistic Regression reached 90.52% accuracy on 9,917 held-out reviews.

Table 27. *Comparison of the three projects*

Aspect	Boston House Price	Car Selling Price	ReviewMind AI
Level	Beginner	Intermediate	Advanced
Task type	Regression	Regression	Sentiment classification, emotion detection, topic modelling
Data size	506 records, 13 features	299 records, 7 selected features	49,582 reviews after cleaning
Test set	102 records	60 records	9,917 reviews
Models	5 regressors	5 baselines + 2 tuned ensembles	TF-IDF + Logistic Regression; DistilBERT; pretrained DistilRoBERTa; NMF
Best result	Random Forest: R^2 0.8752, MAE 2.2140	Gradient Boosting (baseline): R^2 0.7530, MAE 1.1349	Logistic Regression: accuracy 90.52%, ROC-AUC 0.9663
Deliverable	Notebook and reports	Notebook, saved model, reports	Streamlit app, GitHub repository, reports
Key caveat	Imputation before split; single small test set	Test set used in comparison; cars mixed with two-wheelers	Trained and evaluated on IMDb only; DistilBERT trained on a 4,000-review subset

Throughout, the work followed three principles: compare models fairly on the same held-out data, state the conditions behind each result, and report limitations openly rather than over-claiming. The internship strengthened practical skills in data analysis, model evaluation, Python development and application building, and the documented limitations provide a clear agenda for further work.

9. References

[1] Harrison, D. and Rubinfeld, D. L. (1978). Hedonic housing prices and the demand for clean air. Journal of Environmental Economics and Management, 5(1), 81–102.

[2] IMDb Movie Reviews dataset (50,000 reviews). Zenodo. https://zenodo.org/records/7928582

[3] Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., and Potts, C. (2011). Learning word vectors for sentiment analysis. Proceedings of ACL-HLT, 142–150.

[4] Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830. https://scikit-learn.org/stable/

[5] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.

[6] Friedman, J. H. (2001). Greedy function approximation: a gradient boosting machine. Annals of Statistics, 29(5), 1189–1232.

[7] Smola, A. J. and Schölkopf, B. (2004). A tutorial on support vector regression. Statistics and Computing, 14(3), 199–222.

[8] Bergstra, J. and Bengio, Y. (2012). Random search for hyper-parameter optimization. Journal of Machine Learning Research, 13, 281–305.

[9] Sanh, V., Debut, L., Chaumond, J., and Wolf, T. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv:1910.01108. https://arxiv.org/abs/1910.01108

[10] Lee, D. D. and Seung, H. S. (1999). Learning the parts of objects by non-negative matrix factorization. Nature, 401, 788–791.

[11] Wolf, T., et al. (2020). Transformers: State-of-the-art natural language processing. Proceedings of EMNLP: System Demonstrations, 38–45. https://huggingface.co/docs/transformers/

[12] Hartmann, J. Emotion English DistilRoBERTa-base (model card). Hugging Face. https://huggingface.co/j-hartmann/emotion-english-distilroberta-base

[13] McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 56–61.

[14] Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science and Engineering, 9(3), 90–95.

[15] Waskom, M. L. (2021). seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021.

[16] PyTorch documentation. https://pytorch.org/docs/stable/

[17] Streamlit documentation. https://docs.streamlit.io/

[18] ReviewMind AI project repository and README. https://github.com/samarthjain-ai/ShadowFox-ML-Internship

[19] ShadowFox Virtual Internship Program, Internship Offer Letter, 7 October 2026 (Program ID SF-PID-2026-W7613).

[20] Individual project reports and presentations: Boston House Price Prediction (Beginner), Car Selling Price Prediction (Intermediate) and ReviewMind AI (Advanced), November 2026.